

Schema Redesign + Installable React App

A staged plan to separate reusable equipment and parts knowledge from customer-owned assets and service history, while making the existing React application installable from supported browsers.

The target is one durable operational knowledge base and one responsive React/Vite PWA - not duplicated parts research, overwritten history, or a separate React Native product.

Prepared for steering and scope review

July 14, 2026 | Foundational architecture and delivery plan | No implementation approved by this document alone

Executive Summary

The correct destination separates **reusable catalog knowledge**, **specific customer-owned equipment**, and **historical service facts**. The delivery plan also turns the existing responsive React/Vite frontend into an installable, online-first Progressive Web App (PWA) without introducing a second React Native codebase.

Recommended strategy: approve terminology and invariants first; add new structures beside the current schema; reconcile every backfill; transition contracts gradually; deliver installability with conservative cache rules; remove old structures only after stable cutover.

Current-State Diagnosis

Current area	Observed problem	Business consequence
Equipment	Generic model facts and physical-unit facts share one record.	Fifteen similar units can create fifteen separate parts lists.
Compatible parts	Compatibility is keyed by physical EquipmentId.	Fitment knowledge cannot be reused consistently across matching units.
Parts and vendors	A Part points to one Vendor.	Multiple suppliers, branded offers, and substitutes are difficult to represent.
Employee rates	Current rates are mutable Employee fields.	A raise overwrites answers to historical pay and costing questions.
Customer context	Contacts and addresses are embedded.	Multiple roles, effective dates, and historical snapshots are difficult.
Frontend delivery	The React app runs only as a conventional browser site.	Users cannot install and launch it as a standalone work application.

Protected Value

- JobTicketPart and TimeEntry already preserve valuable historical snapshots.
- Technician-safe compatible-part and service-history workflows should remain available through the transition.
- Authorization, soft deletion, explicit enum values, audit records, responsive role-based screens, and /health remain protected boundaries.

Target Domain Model

The model must answer two different questions without confusing them: **what parts are valid for this kind of equipment**, and **what happened to this exact unit**? Equipment configuration answers the first. Equipment asset and installation history answer the second.

SHARED CATALOG	CUSTOMER ASSETS	PARTS + SERVICE
Equipment Manufacturer Equipment Model Equipment Configuration Equipment Part Fitment	Customer / Service Location Equipment Asset Asset Part Installation	Part Vendor Part / Supplier SKU Part Relationship Job Ticket Part Line

Fitment belongs to the configuration. Installation history belongs to the physical asset.

Target Responsibilities

Concept	Responsibility	Historical rule
EquipmentConfiguration	Fitment-relevant year, engine, capacity, trim, or serial range.	Retire configurations rather than mutating identity.
EquipmentAsset	One customer unit with serial, fleet number, ownership, billing, and location.	Service and installation history belongs here.
EquipmentPartFitment	Reviewed configuration, application, and part relationship.	Track source, status, verifier, and date.
PartRelationship	Equivalent, substitute, supersedes, requires, accessory, or kit relationship.	Never infer equivalence merely from shared fitment.
VendorPart	Supplier-specific SKU, current offer, and preference.	Ticket and purchase lines keep snapshots.
AssetPartInstallation	A physical part installed on one asset through a service event.	Track install, removal, replacement, and source ticket.
JobTicketPartLine	Request, approval, ordering, usage, and billing workflow.	A ticket line is not proof of installation.

Customer Context And Historical Rates

Address Structure

- Address stores postal and geographic facts; CustomerAddress supplies billing, mailing, service, and other roles.
- ServiceLocation references an Address while retaining access, safety, gate, and site-specific information.
- Historical work orders and invoices retain address snapshots so current corrections do not rewrite history.

Contact Structure

- Contact represents a person; ContactMethod represents phone, email, or another channel.
- CustomerContact and ServiceLocationContact carry role, primary, notification, billing, and effective-date meaning.
- Tickets may prefill from current contacts, but finalized documents preserve the values used for that transaction.

Operational Notes

Start with explicit CustomerNote, ServiceLocationNote, and EquipmentAssetNote collections. Each note needs category, visibility, effective dates, ShowOnWorkOrder, author, and audit timestamps. Repeated operational rules should later become structured data when the system must enforce them.

Work-order context: assemble visible customer, location, asset, and ticket notes for active work. Snapshot the selected contact, address, and note context at the approved historical-document transition.

Effective-Dated Employee And Billing Rates

- EmployeeCompensationRate records hourly pay and optional internal or burdened cost with non-overlapping effective dates.
- LaborBillingRate remains separate unless steering confirms that customer pricing is truly employee-specific.
- Time-entry creation resolves the applicable rate as of the work date and snapshots source IDs and monetary values.
- Past rates are imported from payroll evidence where available; missing history is marked unknown rather than guessed.

Installable React PWA Scope

The existing React and Vite frontend becomes a Progressive Web App that supported browsers can install directly. It uses the same URLs, API, roles, and responsive screens and launches in a standalone window. **This is not a separate React Native application.**

Initial operating model: online-first. Installation improves launch and application presence, but it does not promise that live ticket data or write workflows work without connectivity.

Initial Delivery Scope

- Standards-based manifest with application name, short name, start URL, scope, theme colors, and standalone display mode.
- Purpose-built 192-pixel, 512-pixel, and maskable application icons.
- Maintained Vite PWA and Workbox integration that generates the production service worker.
- Pre-caching for versioned application-shell assets only.
- Install handling and platform-appropriate guidance where a browser does not expose a programmatic prompt.
- Controlled update messaging that never forces a reload during an active edit or submission.
- Standalone-safe routing, authentication, logout, responsive layouts, and existing geolocation behavior.
- HTTPS hosting, SPA route fallbacks, and explicit cache headers for service-worker and manifest resources.

Platform Boundary

Included	Deferred or separate approval
Desktop and mobile browser installation	React Native and app-store packaging
Standalone launch from one React codebase	A second mobile application codebase
Cached static application shell	Cached authenticated API datasets
Explicit offline and update states	Background write queues and conflict resolution
Existing approved browser capabilities	Push notifications and new native-device integrations

PWA Data Safety, Lifecycle, And Verification

Offline And Cache Boundary

The service worker must not cache authenticated API responses, JWTs, customer records, tickets, attachments, reports, or other business data by default. Static shell caching and browser storage already used by the application are reviewed independently.

- An already-cached application shell may open while offline and must identify that server data is unavailable.
- Only already-approved local draft behavior is preserved; an unsent change is never represented as saved.
- The initial release does not silently replay writes after reconnecting.
- Duplicate submissions are prevented across connectivity changes and update activation.

Security And Lifecycle Controls

- API authorization remains authoritative; installing the application grants no additional access.
- Logout clears application-managed credentials and sensitive local drafts under the approved retention policy.
- Expired-session behavior is defined for installed shells and network reconnection.
- Cache versions remove obsolete shell assets and support recovery from a bad or incompatible service worker.
- Browser and operating-system support is documented without promising identical install prompts everywhere.

Production Verification

Verification area	Required evidence
Installability	Valid manifest and icons; supported desktop/mobile installation checks.
Standalone launch	Installed launch, client routing, login, logout, and role navigation.
Updates	New-version prompt, safe activation, stale-cache cleanup, and recovery path.
Offline state	Clear messaging, no false save confirmation, no automatic write replay.
Cache safety	Credentials and protected API responses absent from Cache Storage.
Regression	Frontend build, unit tests, responsive visual checks, and authentication tests.

Verification uses a production build because development-mode service-worker behavior is intentionally different.

Phased Implementation Plan

Every phase is independently reviewable and additive. Schema stages use forward-only EF Core migrations, preserve authorization, reconcile migrated data, and keep backend build, frontend build, tests, and /health green.

PHASE 0

Approve The Domain And Delivery Contract

Goal: Settle terminology, cardinality, historical rules, and the PWA boundary before implementation.

- Approve the glossary, target ER model, ownership, archive, visibility, and snapshot rules.
- Decide one asset versus multiple assets per ticket and approve rate ownership semantics.
- Approve the PWA platform matrix, online-first boundary, install experience, and update policy.
- Align database design, project scope, API contract, roadmap, system wiki, and runbooks.

Exit gate: Steering explicitly approves the model and the first implementation slice.

PHASE 1

Profile Existing Data

Goal: Understand data quality before automated grouping or backfill.

- Report duplicate equipment, contact, address, and catalog values.
- Identify assets missing model, serial, customer, location, or ownership data.
- Compare parts lists across apparently identical assets and classify ticket-part evidence.
- Inventory employee rates and historical time-entry snapshots.

Exit gate: A reconciliation report exists and ambiguous records are assigned for human review.

PHASE 2

Normalize Customer Context

Goal: Introduce reusable addresses, contacts, and governed notes.

- Add address, contact, relationship, note, visibility, and effective-date structures beside old fields.
- Backfill billing addresses, service-location addresses, and contacts with source IDs retained.
- Add historical document snapshot rules and preserve compatibility during transition.

Exit gate: Migrated customer context reconciles with existing screens and reports.

Phased Implementation Plan

PHASE 3 Introduce Equipment Catalog And Assets

Goal: Separate reusable equipment identity from the physical unit being serviced.

- Add manufacturer, model, configuration, and equipment-asset structures.
- Generate candidate groupings from normalized values and require review for uncertainty.
- Preserve existing equipment APIs through compatibility adapters.

Exit gate: Every active unit is linked to an approved configuration or explicitly unresolved.

PHASE 4 Build The Fitment Matrix

Goal: Store compatibility once at the equipment-configuration level.

- Add candidate, verified, incompatible, and retired fitment records with evidence and audit data.
- Copy per-asset compatible parts as candidates, never as automatically verified facts.
- Expose inherited fitment safely to all matching assets.

Exit gate: One verified fitment reaches every matching asset and no nonmatching configuration.

PHASE 5 Correct The Parts Catalog

Goal: Represent manufactured parts, vendor offers, alternatives, and dependencies independently.

- Separate part identity from supplier-specific VendorPart offers.
- Add reviewed equivalent, substitute, supersedes, requires, accessory, and kit relationships.
- Preserve monetary snapshots on tickets and purchasing records.

Exit gate: A part may have many vendors and alternatives remain explicit rather than inferred.

PHASE 6 Separate Usage From Installation

Goal: Record what was physically installed on one exact asset.

- Add installation lifecycle and replacement-chain records tied to asset and source ticket.
- Backfill only defensible installations; retain ambiguous rows as usage history.
- Apply the approved one-asset or multiple-assets-per-ticket design.

Exit gate: Installation history for one asset never appears for another.

Phased Implementation Plan

PHASE 7 Add Effective-Dated Rates

Goal: Answer historical pay, cost, and billing questions accurately.

- Add non-overlapping compensation and approved billing-rate periods.
- Resolve values as of the work date and snapshot source IDs and amounts on time entries.
- Retire current-rate report fallbacks only after reconciliation.

Exit gate: A rate change affects future calculations without changing prior records.

PHASE 8 Transition APIs And React UI

Goal: Expose corrected concepts while maintaining operating continuity.

- Separate Equipment Catalog from Customer Equipment experiences.
- Add explicit catalog, asset, fitment, relationship, history, context, and rate DTOs.
- Retain compatibility routes temporarily and preserve technician-safe boundaries.

Exit gate: Existing workflows use compatibility contracts while new screens use target contracts.

PHASE 9 Deliver The Installable React PWA

Goal: Make the existing React application safely installable from supported browsers.

- Add manifest, branded icons, Vite PWA integration, and production service-worker registration.
- Cache only versioned shell assets; add install, offline, and controlled-update states.
- Configure HTTPS, SPA fallback, cache headers, platform docs, and production verification.

Exit gate: The app installs and launches standalone, updates predictably, and caches no protected API response.

PHASE 10 Cut Over And Clean Up

Goal: Remove obsolete structures only after reconciliation and a stable release.

- Switch reads and writes, stop compatibility dual-writes, back up production data, and rehearse rollback.
- Remove old fields after the retention window and update all steering and operational documentation.

Exit gate: Reconciliation is clean, monitoring is stable, and rollback has been rehearsed.

Migration, Release, And Estimate Controls

Required Controls

Control	Required behavior
Add before remove	New tables and nullable references arrive before old fields are retired.
No blind merging	Normalized values propose matches; managers confirm ambiguous configurations and fitments.
Preserve source identity	Backfilled records retain their originating IDs and evidence.
Candidates are not facts	Historical use and per-asset lists remain candidates until verified.
Immutable snapshots	Ticket, installation, rate, contact, and address history stays stable.
Forward migrations only	Do not edit historical EF Core migrations.
Compatibility window	Existing consumers remain supported until known callers transition.
Cache only approved assets	Initial service worker caches static shell assets, never protected API responses or credentials.
Controlled client updates	A new frontend version cannot force a reload during active work.
Reconcile every phase	Compare counts, nulls, keys, duplicates, totals, and business scenarios.

Planning Estimate Impact

PWA ADDITION	REVISED COMPLETE PROGRAM
50,000 - 90,000 tokens	550,000 - 890,000 tokens
Manifest, icons, worker behavior, install/update/offline states, hosting, tests, and docs.	Schema redesign plus installable PWA delivery. Planning range, not a billing guarantee.

Actual billed usage depends on model choice, reasoning effort, context reuse, test failures, migration findings, and steering changes.

Decisions And Acceptance Gates

Decisions Required Before Implementation

Decision	Recommended default
One or many assets per ticket	Keep one asset unless actual tickets regularly service multiple units; settle before Phase 6.
Configuration granularity	Split only on attributes that can change fitment.
Equipment customer roles	Name owner, operator/service customer, and billing responsibility explicitly.
Billing-rate ownership	Keep compensation separate and approve pricing rules before table design.
Fitment verification	Restrict verification to Manager/Admin and retain evidence and audit data.
Installable application model	Use one React/Vite PWA; do not create a separate React Native app.
Initial offline behavior	Remain online-first and do not queue or replay writes.
PWA update policy	Notify users and activate the new version at a safe reload point.

Acceptance Criteria

- One verified Frontier fitment is available to every matching asset, while one asset's installation history never appears on another.
- Parts may fit many configurations, have multiple vendor offers, and carry explicit accessory, substitute, or kit relationships.
- Customer context and historical rate, address, contact, and ticket snapshots remain correct after master-data changes.
- Supported browsers install the production React app and launch it in standalone mode with the same routes and authorization.
- Updates do not interrupt active edits; offline state is explicit; protected API responses and credentials never enter Cache Storage.
- Builds, tests, migration reconciliation, backup, rollback rehearsal, responsive checks, and /health all pass.

Recommended first move: complete Phase 0 only. Finish the schema review, settle the PWA platform and online-first boundaries, and approve the target contracts before creating migrations or service-worker changes.

Source Of Record

The complete scope, architecture details, exclusions, evidence list, and phase gates are maintained in docs/proposed-schema-redesign-scope-update-2026-07-14.md.